

Direktori Pada Laptop

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Muhamad Daud Ghifari J0403251026

Haidar Lathif Abqary Setia Harja J0403251097

Adam Mudrika Malik J0403251079

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

Kata Pengantar

Puji syukur kami haturkan kehadiran Allah Swt. yang telah melimpahkan rahmat dan hidayah-Nya sehingga kami bisa menyelesaikan Laporan akhir Project-Based Learning yang berjudul “Direktori Folder Laptop”. Laporan akhir ini disusun sebagai salah satu syarat untuk menyelesaikan tugas akhir semester mata kuliah Algoritma dan Struktur Data.

Tidak lupa juga kami mengucapkan terima kasih kepada dosen yang telah turut memberikan kontribusi dalam penyusunan laporan ini dan kami ucapkan terima kasih juga kepada pihak yang terkait terhadap laporan ini, tidak akan bisa maksimal jika tidak mendapat dukungan dari berbagai pihak.

Penulis menyadari bahwa laporan akhir ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun pembahasannya. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca guna perbaikan di masa mendatang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat, pengetahuan, serta inspirasi bagi para pembaca dan pihak-pihak yang membutuhkan.

Bogor, 26 Juni 2026

DAFTAR ISI

Kata Pengantar	2
DAFTAR ISI	3
BAB I . PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Tujuan Proyek	5
1.3 Ruang Lingkup.....	5
BAB II. ANALISIS DAN PERANCANGAN	6
2.1 Deskripsi Sistem	6
2.3 Algoritma yang Digunakan	8
2.4 Diagram Alir Program (Flowchart).....	9
2.5 Struktur Program.....	10
BAB III. IMPLEMTASI PROGRAM	11
3.1 Tampilan Menu Utama.....	11
3.2 Fitur Create	11
3.3 Fitur Read.....	12
3.4 Fitur Update	12
3.5 Fitur Delete	14
3.6 Fitur <i>Searching</i>	14
3.7 Fitur <i>Sorting</i>	15
3.8 Penyimpanan Data (<i>File Handling</i>).....	15
3.9 Validasi Input dan Error Handling	18
BAB IV. PENGUJIAN DAN ANALISIS	18
4.1 Skenario Pengujian	18
4.2 Kelebihan Sistem	19
4.3 Keterbatasan Sistem.....	19
BAB V. KENDALA DAN SOLUSI	20
BAB VI. KESIMPULAN DAN SARAN	20
Kesimpulan	20
Saran Berdasarkan keterbatasan sistem yang ada saat ini, berikut adalah beberapa fitur yang dapat dikembangkan pada masa mendatang:	21
DAFTAR PUSTAKA.....	22

LAMPIRAN.....	23
Lampiran A. Link Github.....	23
Lampiran B. Screenshoot Program	23
Lampiran C. Struktur Folder Proyek	27
Lampiran D. Commit History GitHub	27

BAB I . PENDAHULUAN

1.1 Latar Belakang

Direktori pada komputer/laptop adalah tempat yang digunakan komputer untuk menyimpan dan mengelompokkan file. Direktori atau folder bekerja dengan menghubungkan berbagai jenis file dan menampilkannya dalam satu panel sehingga memudahkan pengguna untuk mengakses dan mengolah file dalam komputer. Direktori juga dilengkapi dengan path yang berfungsi memberitahu pengguna lokasi persis sebuah file berada. Direktori diimplementasikan untuk membantu pengguna dalam menavigasikan lokasi file serta mengelompokkan file berdasarkan fungsi dan sumber dari file tersebut.

Direktori pada laptop memiliki struktur mekanisme yang sama dengan graph berbentuk linked tree dengan bagian pathnya memiliki struktur stack. Melihat fenomena ini, penulis berinisiatif untuk membuat sebuah simulasi rekayasa pada python yang mengimplementasikan mekanisme serupa dengan direktori pada laptop menggunakan konsep linked tree dan stack. Tujuan utama dari pembuatan simulasi rekayasa ini adalah untuk memperdalam pemahaman penulis mengenai pembuatan linked tree, stack, serta proses implementasi ilmu dalam bahasa pemrograman.

1.2 Tujuan Proyek

1. Membuat simulasi direktori folder berbasis python CLI
2. Menyediakan ruang uji coba perintah terminal yang aman bagi pemula.
3. Mengimplementasikan konsep struktur data *tree* dan *stack* ke dalam kode pemrograman.

1.3 Ruang Lingkup

- Platform CLI (Console)
- Bahasa Python
- Penyimpanan file JSON
- Fitur CRUD
- Searching
- Sorting

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem yang dibuat dalam proyek tugas akhir semester ini adalah sebuah aplikasi simulasi manajemen direktori *folder* berbasis *Command Line Interface* (CLI) dengan menggunakan bahasa pemrograman Python. Sistem ini dirancang untuk meniru perilaku dan operasi dasar dari sistem pengelolaan file pada sistem operasi asli, namun bekerja secara terisolasi di dalam memori komputer sehingga aman digunakan untuk simulasi.

Untuk merepresentasikan struktur direktori yang bersifat hierarkis dan bercabang, sistem ini mengimplementasikan struktur data *tree*. Dalam struktur ini, setiap *folder* bertindak sebagai sebuah *node* yang dapat menampung *sub-node* berupa file maupun *folder* lain di dalamnya. Selain itu, sistem juga menerapkan struktur data *stack* yang berfungsi untuk menyimpan riwayat navigasi pengguna (*path*). Ketika pengguna berpindah dari satu direktori ke direktori lain, posisi direktori sebelumnya akan disimpan ke dalam *stack*. Untuk menjaga keberlanjutan data agar tidak hilang saat aplikasi ditutup, sistem ini menggunakan file berformat JSON (*JavaScript Object Notation*) sebagai media penyimpanan data persisten.

Sistem simulasi direktori ini menyediakan beberapa fitur utama yang dapat dieksekusi oleh pengguna melalui perintah teks, yaitu:

1. **Tambah File/Folder:** Memungkinkan pengguna untuk membuat entitas file baru atau *folder* baru di dalam direktori yang sedang aktif.
2. **Masuk ke Folder:** Memungkinkan pengguna untuk melakukan navigasi dan berpindah masuk ke dalam *sub-folder* yang ada di dalam direktori aktif.
3. **Kembali ke *Parent*:** Fitur untuk keluar dari *folder* aktif dan kembali ke direktori satu tingkat di atasnya (*parent directory*), di mana proses pemanggilan kembali posisi direktori ini menggunakan riwayat yang tersimpan di dalam *stack*.
4. **Hapus File/Folder:** Memungkinkan pengguna untuk menghapus file atau *folder* yang ada di dalam direktori yang sedang diakses.
5. **Lihat Isi Folder:** Berfungsi untuk menampilkan daftar seluruh file dan *folder* yang berada di dalam direktori yang sedang diakses.
6. **Ganti Nama File/Folder:** Memungkinkan pengguna untuk mengubah nama dari file atau *folder* yang sudah ada tanpa mengubah posisinya dalam struktur.

7. **Cari Nama File/Folder:** Fitur pencarian untuk menemukan lokasi semua file atau *folder* yang memiliki nama mirip maupun sama persis di seluruh direktori berdasarkan kata kunci yang dimasukkan.
8. **Simpan ke JSON:** Berfungsi untuk melakukan sinkronisasi dan menyimpan seluruh perubahan struktur data *tree* yang ada di memori ke dalam file penyimpanan JSON secara permanen.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi
<i>Tree</i>	Struktur data utama yang berisi keseluruhan file dan folder.
<i>Stack</i>	Menyimpan Riwayat navigasi <i>path</i> agar pengguna dapat Kembali ke parent.
<i>Dictionary</i>	menyimpan data atribut dan hubungan hierarki objek (file dan folder) untuk memudahkan proses pemetaan dan penyimpanan data ke dalam format JSON.

Alasan Pemilihan Struktur Data

Struktur data *tree* dipilih karena karakteristik hierarkinya merepresentasikan struktur direktori folder pada sistem operasi secara persis melalui hubungan induk-anak (*parent-child relationship*).

Stack dipilih karena menerapkan prinsip *Last-In, First-Out* (LIFO); dalam implementasinya, ketika pengguna menavigasi masuk ke sebuah *folder*, posisi direktori saat ini akan disimpan (*push*) ke dalam *stack* sehingga sistem dapat dengan mudah memanggilnya kembali (*pop*) saat pengguna ingin kembali ke folder sebelumnya (*parent directory*).

Dictionary didasarkan pada kesamaan strukturnya yang berbasis pasangan kunci-nilai (*key-value pairs*), yang selaras dengan format JSON sehingga mempermudah proses pemetaan dan penyimpanan data secara persisten.

2.3 Algoritma yang Digunakan

Searching

- Depth-First Search

Alasan Penggunaan :

Algoritma *Depth-First Search* (DFS) dipilih karena kelebihanannya dalam menelusuri kedalaman ruang pencarian secara cepat dengan alokasi memori yang relatif kecil (Lailiyah et al., 2017). Sistem kerja DFS ini melakukan pencarian dengan cara menyusuri satu cabang direktori secara mendalam terlebih dahulu hingga mencapai ujung folder (*dead-end*) sebelum kembali mundur (*backtrack*) untuk memeriksa cabang lainnya, karena itu DFS ini kami gunakan karena sesuai dengan tujuan kami yaitu mencari semua file dan folder yang sesuai dengan kata kunci.

Sorting

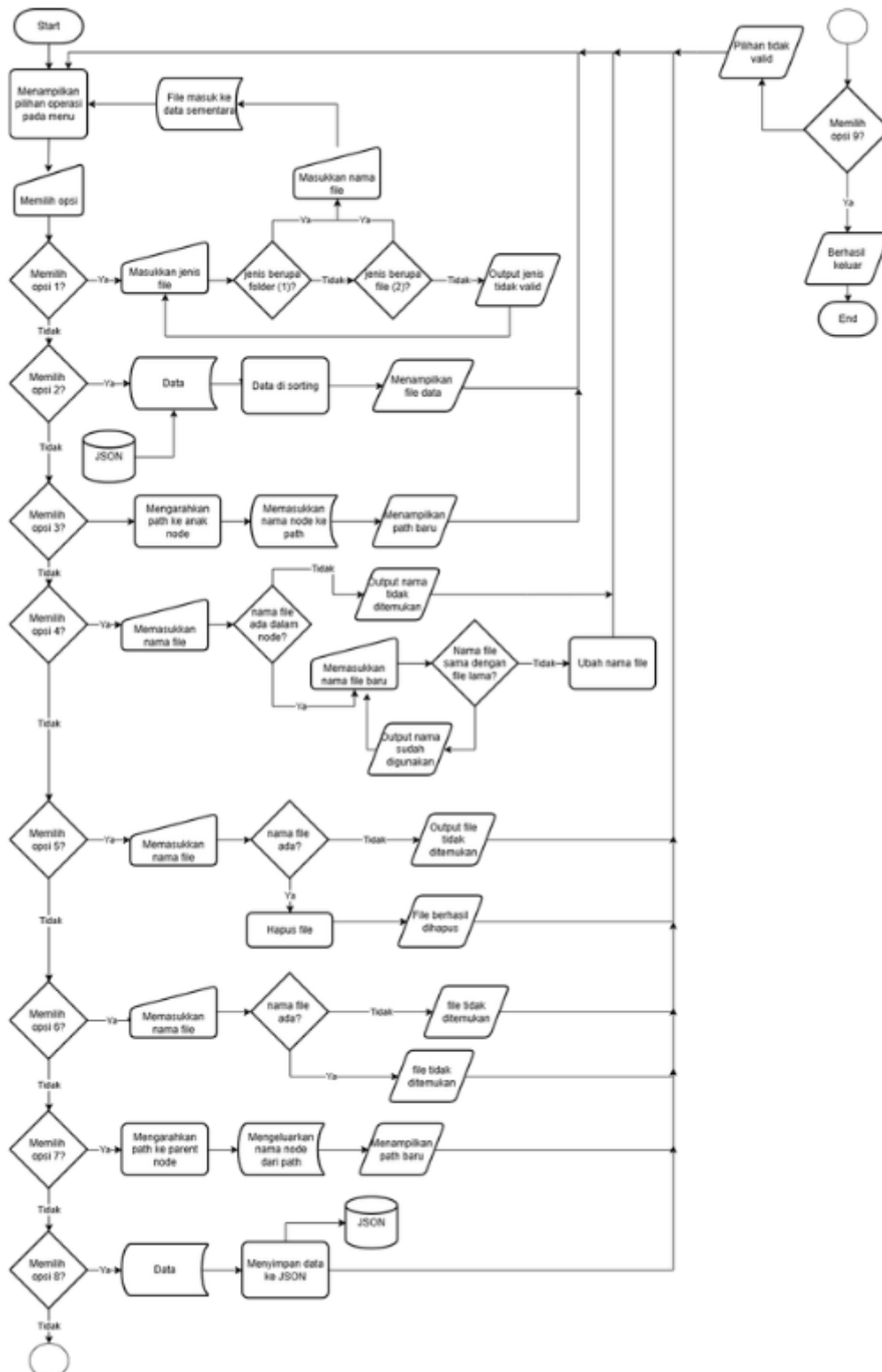
- Tim Sort

Alasan penggunaan:

Pemilihan algoritma Timsort sebagai metode pengurutan nama file dan folder dalam sistem bukan hanya karena ini merupakan fungsi bawaan python (`sorted`) tetapi juga memiliki keunggulan lain, seperti Timsort memang dirancang untuk mengenali pola data yang sudah terurut sebagian secara alami. Menurut Wibowo & Faisal (2024) "Timsort dirancang untuk memanfaatkan urutan elemen berurutan (*run*) yang sudah ada di sebagian besar data dunia nyata... Keuntungan dari menggabungkan urutan *run* dibandingkan dengan menggabungkan sub-daftar berukuran tetap (seperti yang dilakukan Merge Sort klasik) adalah bahwa cara ini mengurangi jumlah total perbandingan yang diperlukan untuk mengurutkan seluruh daftar."

Dalam konteks aplikasi simulasi direktori, data nama file maupun folder yang dimuat dari file JSON sering kali sudah berada dalam kondisi terurut atau semi-terurut. Kemampuan Timsort dalam meminimalkan jumlah operasi perbandingan string ini memastikan proses pemuatan dan penampilan direktori pada antarmuka CLI berjalan secara instan dan lebih hemat memori.

2.4 Diagram Alir Program (Flowchart)



Gambar 1 Flowchar Program

2.5 Struktur Program

- main.py

File ini berfungsi sebagai modul utama dan pusat alur eksekusi dari keseluruhan sistem simulasi ini. Dalam file ini, seluruh komponen logika program akan diintegrasikan, mulai dari inisialisasi struktur data *tree* dan *stack*, manajemen navigasi, pemanggilan dan penyimpanan data file JSON. File ini menyediakan CLI yang interaktif bagi pengguna dengan memetakan input perintah teks ke dalam fitur-fitur utama sistem ini yang meliputi membuat, menghapus file/folder, masuk ke direktori/folder, kembali ke parent, ubah nama, menampilkan isi direktori secara terurut, dan fitur pencarian nama file/folder.

- dataMemori.json

File ini berfungsi untuk menyimpan data secara permanen. Fungsi file json ini ada 2 meliputi:

1. **Penyimpanan Struktur Data (*Data Saving*)**
Menyimpan seluruh data hierarki direktori (nama file/folder, tipe, waktu di ubah, children jika bertipe folder) yang telah dibuat oleh pengguna ke dalam bentuk file fisik berformat teks.
2. **Rekonstruksi Data (*Data Loading*)**. Berfungsi sebagai sumber acuan bagi program saat pertama kali dijalankan untuk memuat (*load*) kembali data lama, sehingga struktur direktori yang muncul pada CLI persis seperti kondisi terakhir sebelum aplikasi ditutup.

BAB III. IMPLEMTASI PROGRAM

3.1 Tampilan Menu Utama

```
Path: home

Menu:
1. Buat Folder/file
2. Lihat Isi Folder home
3. Masuk ke Folder
4. Ganti Nama
5. Hapus File/Folder
6. Cari File/Folder
7. Kembali
8. Simpan ke Json
9. Keluar
=====
Pilih: █
```

Gambar 2 Tampilan Menu Utama

3.2 Fitur Create

Langkah-langkah fitur create:

1. Memilih menu create (nomor 1)
2. Masukan tipe yang ingin dibuat 1(Folder) dan 2(File), jika salah akan menampilkan salah dan menginput ulang, jika benar akan lanjut
3. Masukan nama file/folder, jika sudah ada maka akan menampilkan nama sudah digunakan dan harus memasukan ulang, jika benar maka file/folder berhasil dibuat.

```

Pilih: 1

===Pilih tipe===
[1] FOLDER
[2] FILE
Pilih tipe (1/2): 4
Tipe salah. Silahkan input ulang
Pilih tipe (1/2): 1
Masukan nama Folder: Algoritmas
Nama sudah digunakan!, Silahkan masukan ulang
Masukan nama Folder: Algoritma
Folder berhasil dibuat.

```

Gambar 3 Fitur Create

3.3 Fitur Read

Langkah-langkah fitur read:

1. Pengguna memilih menu read(2)
2. Sistem akan menampilkan isi dari direktori saat ini dengan format seperti pada gambar

```

Pilih: 2
===Isi Folder home===
[📁] & C:/Users/HP/AppData/... 03/06/2026 15:56 PM
[📁] Algoritma                  26/06/2026 17:06 PM
[📁] Algoritmas                 03/06/2026 14:16 PM
[📁] Alias                     03/06/2026 13:55 PM
[📁] ava                       03/06/2026 15:57 PM
[📄] Data.txt                   03/06/2026 11:12 AM
[📄] Samyang.png               03/06/2026 13:55 PM
[📄] Scoantil.txt              03/06/2026 14:31 PM

Path: home

```

Gambar 4 Fitur Read

3.4 Fitur Update

Langkah-langkah fitur update:

1. Pengguna memilih menu update(4).

2. Sistem akan menampilkan isi direktori saat ini.
3. Pengguna memasukan nama yang ingin diubah dan nama barunya.
4. Jika nama baru sudah digunakan atau nama yang ingin diubah tidak ditemukan maka akan kembali ke menu utama dan mengulang dari langkah pertama.
5. Jika sudah sesuai maka nama berhasil diubah.

```
=====
Pilih: 4

Path:  home

====Daftar Folder====
[📁] Algoritmas
[📄] Data.txt
[📁] Aliasi
[📄] Samyang.png
[📄] Scoantil.txt
[📁] & C:/Users/HP/AppData/...
[📁] ava

Nama yang ingin diubah: Algoritmas
Nama baru: Aliasi
Nama sudah digunakan!
```

Gambar 5 Fitur Update

```
Nama yang ingin diubah: Algo
Nama baru: Algorit
Folder/file tidak ditemukan
```

Gambar 6 Fitur update jika tidak ditemukan

```
Nama yang ingin diubah: Algoritmas
Nama baru: Algoritma
Berhasil mengganti nama.
```

Gambar 7 Fitur Update Jika Berhasil

3.5 Fitur Delete

Langkah-langkah fitur delete:

1. Pengguna memilih menudelete(5).
2. Sistem akan menampilkan isi direktori saat ini yang sudah terurut.
3. Pengguna memasukan nama yang ingin dihapus.
4. Jika nama tidak ditemukan maka akan kembali ke menu utama dan mengulang dari langkah pertama.
5. Jika sudah sesuai maka nama berhasil dihapus.

```
Pilih: 5
===Isi Folder home===
[📁] & C:/Users/HP/AppData/... 03/06/2026 15:56 PM
[📁] Algoritmas                  03/06/2026 14:16 PM
[📁] Alias                      03/06/2026 13:55 PM
[📁] ava                        03/06/2026 15:57 PM
[📄] Data.txt                   03/06/2026 11:12 AM
[📄] Samyang.png               03/06/2026 13:55 PM
[📄] Scoantil.txt              03/06/2026 14:31 PM
Masukan Nama yang ingin dihapus: Algo
Folder/File tidak ditemukan.
```

Gambar 8 Fitur Delete Jika Tidak Ditemukan

```
Masukan Nama yang ingin dihapus: Algoritmas
Berhasil dihapus
```

Gambar 9 Fitur Delete Jika Berhasil

3.6 Fitur Searching

Langkah-langkah fitur searching:

1. Pengguna memilih menu cari(6):
2. Pengguna memasukan kata kunci
3. Sistem akan mencari keseluruhan tree dan akan menampilkan nama file maupun folder yang sesuai(menggunakan logika 'in')

4. Menampilkan dengan format Ditemukan: nama file/folder [*path* dari file/folder tersebut]

```
Pilih: 6
Masukkan kata kunci: Algoritma
Ditemukan: Algoritma [Path: home/Aliasi/Algoritma]
Ditemukan: Algoritmas.txt [Path: home/Aliasi/Algoritmas.txt]
Ditemukan: & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/Documents/Tugas
uktur Data/ASD_15_File Reader/ASD-15-Direktori-Folder-Laptop/main.py" [Path: home/& C:/Users/HP/AppData
.exe "c:/Users/HP/Documents/Tugas (kuliah)/Semester 2/Algoritma dan Struktur Data/ASD_15_File Reader/AS
Ditemukan: Algoritma [Path: home/Algoritma]
Ditemukan: Algoritmas [Path: home/Algoritmas]
```

Gambar 10 Fitur Searching

3.7 Fitur Sorting

Fitur ini digunakan otomatis tanpa harus memilih menu tertentu, fitur ini digunakan ketika akan menampilkan daftar isi sebuah folder dengan menggunakan Tim Sort dan fungsi bawaan dari python dan menggunakan lambda x: [x.name.lower](#) agar menyortir berdasarkan nama lowercasenya(tidak *case sensitive*)

```
Pilih: 2
===Isi Folder home===
[Folder] & C:/Users/HP/AppData/... 03/06/2026 15:56 PM
[Folder] Algoritma 26/06/2026 17:06 PM
[Folder] Algoritmas 03/06/2026 14:16 PM
[Folder] Aliasi 03/06/2026 13:55 PM
[Folder] ava 03/06/2026 15:57 PM
[File] Data.txt 03/06/2026 11:12 AM
[File] Samyang.png 03/06/2026 13:55 PM
[File] Scoantil.txt 03/06/2026 14:31 PM
Path: home
```

Gambar 11 Fitur Sorting

```
ASD-15-Direktori-Folder-Laptop - main.py
1 sorted_children = sorted(self.current.children, key=lambda x: x.name.lower())
```

Gambar 12 Kode Program Fitur Sorting

3.8 Penyimpanan Data (*File Handling*)

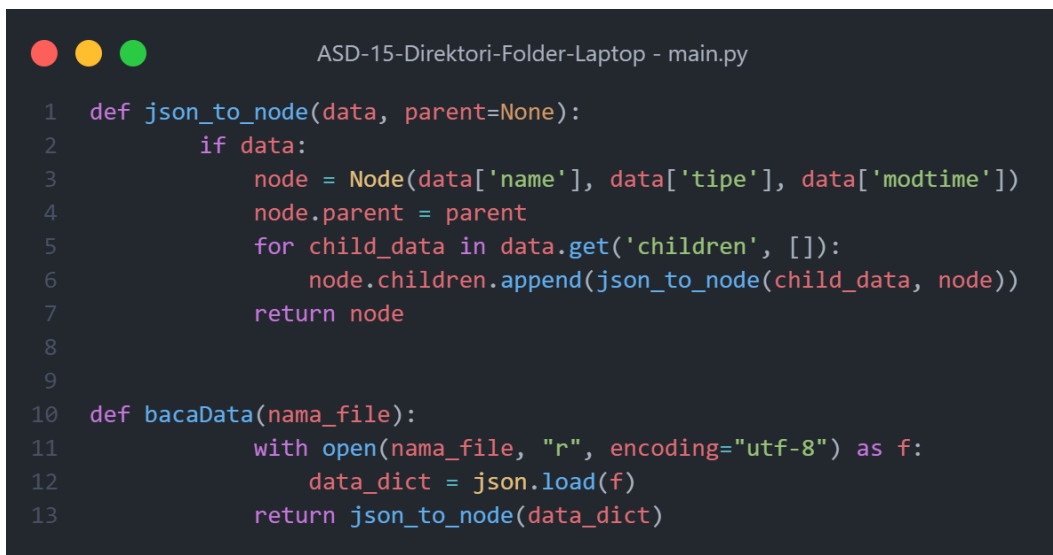
1. Format File

Sistem simulasi direktori ini menggunakan format file JSON dengan ekstensi .json sebagai media penyimpanan data lokal secara permanen. Format JSON dipilih karena strukturnya yang berbasis teks ringan dan menggunakan format pasangan kunci-nilai (*key-value pairs*).

Karakteristik data terstruktur pada JSON memiliki logika yang sama dengan tipe data *dictionary* pada Python, sehingga ideal untuk struktur hierarki *tree* dari direktori *folder* dan *file*.

2. Cara membaca data

Proses pembacaan data dilakukan melalui mekanisme **deserialisasi**, yaitu membaca kembali data teks dari file JSON untuk di ubah menjadi objek struktur data *tree* (node) di dalam program. Pertama membaca data dengan “with open ‘r’”. lalu akan mengirimkan nya ke fungsi `json_to_node`. Di fungsi ini data json akan diubah menjadi node dengan format yang sesuai, jika memiliki isi(bertipe folder) seperti maka akan menggunakan perulangan dan fungsi rekursif untuk mengambil semua isi hingga bagian terdalam.



```
ASD-15-Direktori-Folder-Laptop - main.py
1 def json_to_node(data, parent=None):
2     if data:
3         node = Node(data['name'], data['tipe'], data['modtime'])
4         node.parent = parent
5         for child_data in data.get('children', []):
6             node.children.append(json_to_node(child_data, node))
7         return node
8
9
10 def bacaData(nama_file):
11     with open(nama_file, "r", encoding="utf-8") as f:
12         data_dict = json.load(f)
13         return json_to_node(data_dict)
```

Gambar 13 Kode Program Cara Membaca Data

3. Cara menyimpan data

Ini menggunakan cara sama persis seperti membaca data, tetapi kebalikannya yaitu menggunakan fungsi simpan yang data *dictionary*nya diambil dengan menggunakan fungsi `node_ke_dict` untuk mengonversi data node menjadi *dictionary* dan akan rekursif juga jika data memiliki *children*


```

ASD-15-Direktori-Folder-Laptop - main.py

1  def node_ke_dict(self, node):
2      data = {
3          "name" : node.name,
4          "tipe" : node.tipe,
5          "modtime" : node.modtime,
6          "children" : [self.node_ke_dict(child) for child in node.children]
7      }
8      if node.parent is not None:
9          data["parent"] = node.parent.name
10
11     return data
12
13  def simpan_json(self, filename):
14      data_dict = self.node_ke_dict(self.root)
15
16      with open(filename, "w", encoding="utf-8") as f:
17          json.dump(data_dict, f, indent=4)
18      print("Berhasil simpan data")

```

Gambar 14 Kode Program Cara Menyimpan Data

```

{
  "name": "home",
  "tipe": "Folder",
  "modtime": "03/06/2026 11:12 AM",
  "children": [
    {
      "name": "Data.txt",
      "tipe": "File",
      "modtime": "03/06/2026 11:12 AM",
      "children": [],
      "parent": "home"
    },
    {
      "name": "Aliasi",
      "tipe": "Folder",
      "modtime": "03/06/2026 13:55 PM",
      "children": [
        {
          "name": "i",
          "tipe": "Folder",
          "modtime": "03/06/2026 14:15 PM",
          "children": [],
          "parent": "Aliasi"
        },
        {
          "name": "Algoritma",
          "tipe": "Folder",
          "modtime": "26/06/2026 17:30 PM",
          "children": [],
          "parent": "Aliasi"
        }
      ]
    }
  ]
}

```

Gambar 15 Contoh Isi File JSON

3.9 Validasi Input dan Error Handling

Kesalahan input

Nama tidak ditemukan(buat file/folder)

Nama sudah digunakan(buat file/folder)

Nama sudah digunakan(selain buat folder/file)

Nama tidak ditemukan(selain buat folder/file)

Penanganan

Program meminta input ulang

Program meminta input ulang

Program akan kembali ke menu utama

Program akan kembali ke menu utama

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Hasil
1	Tambah data	Berhasil. Sistem mampu membuat file/folder baru dengan validasi input yang jelas
2	Navigasi data	Berhasil. Sistem mampu berpindah dari direktori saat ini kedalam sub-foldernya maupun dari sub-folder ke folder induknya
3	Edit data	Berhasil. Sistem mampu mengubah nama file maupun folder yang dipilih tanpa ada duplikasi nama dalam direktori tersebut dan tidak merusak relasi dengan sub-foldernya
4	Hapus data	Berhasil. Sistem mampu menghapus objek data tunggal atau folder beserta sub-folder didalamnya
5	Sorting data	Berhasil. Sistem secara otomatis akan menampilkan nama file dan folder yang sudah tersortir secara ascending dengan bersifat <i>case-insensitive</i> menggunakan fungsi bawaan
6	Searching data	Berhasil. Sistem mampu mencari dan menemukan semua file/folder dari root hingga semua cabang terdalam yang namanya mirip atau sama dengan kata kunci yang dimasukan oleh user menggunakan algoritma <i>Depth-First Search</i> .
7	Penyimpanan data	Berhasil. Sistem mampu melakukan serialisasi data ke file json saat menyimpan, dan melakukan deserialisasi data saat menjalankan program

4.2 Kelebihan Sistem

Berdasarkan hasil perancangan dan pengujian yang telah dilakukan, sistem simulasi direktori berbasis CLI ini memiliki beberapa kelebihan utama, antara lain:

1. **Penyimpanan Data Bersifat Persisten (Permanen)** Sistem tidak hanya berjalan sementara di dalam memori volatilitas (RAM). Dengan menggunakan modul JSON, seluruh perubahan yang dilakukan pengguna akan diserialisasikan ke dalam file json. Hal ini menjamin data tetap utuh dan dapat dimuat kembali secara utuh saat program dijalankan ulang.
2. **Stabilitas Program yang Tinggi** Program dilengkapi dengan mekanisme penanganan kesalahan (*error handling / exception*) yang baik, seperti validasi input perintah, deteksi nama ganda (*duplicate name*), serta pembatasan akses navigasi. Hal ini membuat aplikasi tangguh dan tidak mudah keluar secara paksa meskipun menerima input ilegal dari pengguna.
3. **Keamanan terhadap Sistem Operasi Asli** Program ini murni berjalan di atas simulasi struktur data *tree* buatan di dalam lingkup program itu sendiri. Setiap perintah pembuatan, pengubahan, maupun penghapusan berkas tidak akan memengaruhi atau mengganggu struktur direktori dan file asli yang ada pada sistem operasi komputer pengguna.

4.3 Keterbatasan Sistem

Di samping kelebihan yang dimiliki, sistem ini masih memiliki beberapa keterbatasan yang dapat dijadikan acuan untuk pengembangan lebih lanjut:

1. **Belum Mendukung Sistem Manajemen Basis Data** Penyimpanan data masih mengandalkan file flat JSON lokal. Sistem belum terintegrasi dengan database relasional (seperti MySQL/PostgreSQL) ataupun non-relasional (seperti MongoDB), sehingga kurang optimal jika digunakan untuk menangani manajemen data dalam skala yang sangat besar (*enterprise scale*) atau bersifat multi-user.
2. **Keterbatasan Format Informasi File** Program ini baru menyimulasikan struktur hierarki nama berkas, tipe, dan waktu modifikasi saja. Sistem belum mendukung penyimpanan konten atau isi teks di dalam file sesungguhnya, serta belum mengimplementasikan sistem perizinan hak akses file (*file permission* seperti read, write, execute).

3. **Ketergantungan pada Struktur Data Tunggal di Memori** Karena menggunakan struktur data *tree* kustom, seluruh node harus dimuat ke dalam memori RAM saat program aktif (proses deserialisasi). Jika data direktori berkembang hingga jutaan entitas, pendekatan ini berpotensi meningkatkan beban memori komputer secara signifikan pada saat dijalankan.

BAB V. KENDALA DAN SOLUSI

No	Kendala	Kendala
1	Data hilang saat program ditutup	Menggunakan file json
2	Program error saat input tidak sesuai	Menggunakan Validasi logika
3	Program tidak berjalan sesuai harapan	Melakukan perbaikan di beberapa fitur
4	Logika program salah(data tidak masuk saat membaca data dan mengirimkan data json)	Melakukan proses serialisasi dan deserialisasi

BAB VI. PEMBAGIAN TUGAS ANGGOTA

NAMA	TUGAS
Muhamad Daud Ghifari	Pembuatan File Handling(serialisasi deserialisasi), pengembangan CRUD, pembuatan fitur masuk folder/direktori.
Haidar Lathif Abqary Setia Harja	Waktu update/dibuat file/folder, perbaikan File Handling bagian waktu dibuat, melengkapi fitur create, fitur Search.
Adam Mudrika Malik	Keseluruhan program awal(base program)

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

1. **Keberhasilan Implementasi Fitur dan Navigasi Utama** Proyek ini telah berhasil membangun sebuah sistem simulasi manajemen direktori yang fungsional di dalam file [main.py](#). Program mampu mengeksekusi seluruh fungsi dasar manipulasi data secara *real-time*, meliputi pembuatan (file/folder), perubahan nama (*rename*), penghapusan, pengurutan abjad otomatis yang efisien menggunakan Timsort, hingga pencarian mendalam berbasis algoritma *Depth-First Search* (DFS).

2. **Keberhasilan Struktur Hierarki dan Simulasi yang Aman** Penerapan struktur data *Tree* telah teruji dalam memodelkan hubungan hierarki simpul (*node*) antara induk (*parent*) dan anak (*children*) secara logis untuk merepresentasikan folder dan file. Simulasi ini berhasil beroperasi secara stabil di dalam ruang isolasi program tanpa mengganggu atau merusak struktur direktori asli pada sistem operasi komputer pengguna.
3. **Keberhasilan Penanganan Penyimpanan Data Persisten** Proyek ini berhasil mengatasi kendala hilangnya data dengan menerapkan proses serialisasi dan deserialisasi secara rekursif. Dengan mengonversi objek *Tree Node* menjadi tipe data *Dictionary* sebelum disimpan ke file JSON, program sukses menjaga konsistensi data sehingga seluruh perubahan status direktori tersimpan secara permanen dan dapat dimuat kembali secara utuh saat Program dijalankan ulang.

Saran

Berdasarkan keterbatasan sistem yang ada saat ini, berikut adalah beberapa fitur yang dapat dikembangkan pada masa mendatang:

1. **Peningkatan Konsistensi dalam Validasi Input**
Fitur ini akan dikembangkan dengan memperketat dan menyamakan standar validasi pada setiap input perintah dari pengguna. Hal ini meliputi pembatasan karakter spesial dalam penamaan folder atau file, penanganan spasi berlebih, serta pencegahan input kosong pada seluruh fitur guna menghindari kegagalan logis pada struktur data *tree*.
2. **Konsistensi Pengurutan (*Sorting*) saat Menampilkan Isi**
Perlu diterapkan standar aturan pengurutan (*sorting*) yang konsisten di setiap fungsi yang menampilkan daftar direktori. Proses pengurutan menggunakan Timsort sebaiknya tidak hanya berjalan pada perintah dasar menampilkan isi, tetapi juga otomatis diterapkan secara seragam pada hasil keluaran fitur pencarian (DFS) dan fitur manipulasi data lainnya agar informasi selalu tersaji secara ascending.
3. **Pengembangan Fitur Navigasi Langsung (Menuju *Root* atau *Path* Spesifik)** Sistem navigasi yang saat ini masih terbatas pada perpindahan satu tingkat ke folder induk (`cd ..`) perlu ditingkatkan fungsinya. Pengembangan fitur ini bertujuan untuk menambahkan fitur navigasi langsung yang memungkinkan pengguna melompat kembali ke direktori utama secara instan (`cd /` atau `cd ~`) atau berpindah langsung menuju alamat *path* folder yang spesifik tanpa harus masuk satu per satu.

4. **Optimalisasi Tampilan Antarmuka CLI yang Berwarna dan Berstruktur** Untuk meningkatkan pengalaman pengguna (*User Experience*), tampilan antarmuka berbasis teks (CLI) dapat dikembangkan agar lebih menarik dan mudah dipahami. Hal ini dapat dicapai dengan menambahkan pewarnaan teks khusus (misalnya membedakan warna teks antara jenis folder dan file) serta memanfaatkan karakter grafis teks untuk membentuk visualisasi struktur pohon direktori yang lebih jelas dan hierarkis.

DAFTAR PUSTAKA

- Lailiyah, S., Yusnita, A., & Panotogomo, T. A. (2017). Penerapan algoritma depth first search pada sistem pencarian dokumen. *Prosiding SNITT Poltekba*, 2, 174–179. <https://jurnal.poltekba.ac.id/index.php/prosiding/article/view/395>
- Wibowo, F. R., & Faisal, M. (2024). Comparative analysis of sorting algorithms: TimSort Python and classical sorting methods. *JISA (Jurnal Informatika dan Sains)*, 7(1), 14–15. <https://doi.org/10.30813/jisa.v7i1.1785>

LAMPIRAN

Lampiran A. Link Github

<https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop>

Lampiran B. Screenshoot Program

```
Path: home

Menu:
1. Buat Folder/file
2. Lihat Isi Folder home
3. Masuk ke Folder
4. Ganti Nama
5. Hapus File/Folder
6. Cari File/Folder
7. Kembali
8. Simpan ke Json
9. Keluar
=====
Pilih: █
```

```
Pilih: 1

===Pilih tipe===
[1] FOLDER
[2] FILE
Pilih tipe (1/2): 4
Tipe salah. Silahkan input ulang
Pilih tipe (1/2): 1
Masukan nama Folder: Algoritmas
Nama sudah digunakan!, Silahkan masukan ulang
Masukan nama Folder: Algoritma
Folder berhasil dibuat.
```

Pilih: 2

===Isi Folder home===

[📁]	& C:/Users/HP/AppData/...	03/06/2026 15:56 PM
[📁]	Algoritma	26/06/2026 17:06 PM
[📁]	Algoritmas	03/06/2026 14:16 PM
[📁]	Aliasi	03/06/2026 13:55 PM
[📁]	ava	03/06/2026 15:57 PM
[📄]	Data.txt	03/06/2026 11:12 AM
[📄]	Samyang.png	03/06/2026 13:55 PM
[📄]	Scoantil.txt	03/06/2026 14:31 PM

Path: home

Pilih: 3

===Isi Folder home===

[📁]	& C:/Users/HP/AppData/...	03/06/2026 15:56 PM
[📁]	Algoritma	26/06/2026 17:29 PM
[📁]	Algoritmas	26/06/2026 17:29 PM
[📁]	Aliasi	03/06/2026 13:55 PM
[📁]	ava	03/06/2026 15:57 PM
[📄]	Data.txt	03/06/2026 11:12 AM
[📄]	Samyang.png	03/06/2026 13:55 PM
[📄]	Scoantil.txt	03/06/2026 14:31 PM

Masuk: Algoritma

Path: home/Algoritma


```
=====
Pilih: 4
```

```
Path: home
```

```
====Daftar Folder====
```

```
[📁] Algoritmas
[📄] Data.txt
[📁] Aliasi
[📄] Samyang.png
[📄] Scoantil.txt
[📁] & C:/Users/HP/AppData/...
[📁] ava
```

```
Nama yang ingin diubah: Algoritmas
```

```
Nama baru: Aliasi
```

```
Nama sudah digunakan!
```

```
Nama yang ingin diubah: Algoritmas
```

```
Nama baru: Algoritma
```

```
Berhasil mengganti nama.
```

```
=====
Pilih: 5
```

```
===Isi Folder home===
```

```
[📁] & C:/Users/HP/AppData/... 03/06/2026 15:56 PM
[📁] Algoritmas                03/06/2026 14:16 PM
[📁] Aliasi                    03/06/2026 13:55 PM
[📁] ava                      03/06/2026 15:57 PM
[📄] Data.txt                  03/06/2026 11:12 AM
[📄] Samyang.png              03/06/2026 13:55 PM
[📄] Scoantil.txt             03/06/2026 14:31 PM
```

```
Masukan Nama yang ingin dihapus: Algo
```

```
Folder/File tidak ditemukan.
```

```
Masukan Nama yang ingin dihapus: Algoritmas
```

```
Berhasil dihapus
```

```
Pilih: 6
Masukkan kata kunci: Algoritma
Ditemukan: Algoritma [Path: home/Aliasi/Algoritma]
Ditemukan: Algoritmas.txt [Path: home/Aliasi/Algoritmas.txt]
Ditemukan: & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/Documents/Tugas
struktur Data/ASD_15_File Reader/ASD-15-Direktori-Folder-Laptop/main.py" [Path: home/& C:/Users/HP/AppData
.exe "c:/Users/HP/Documents/Tugas (kuliah)/Semester 2/Algoritma dan Struktur Data/ASD_15_File Reader/AS
Ditemukan: Algoritma [Path: home/Algoritma]
Ditemukan: Algoritmas [Path: home/Algoritmas]
```

```
Path: home/Algoritma
```

```
Menu:
```

1. Buat Folder/file
2. Lihat Isi Folder Algoritma
3. Masuk ke Folder
4. Ganti Nama
5. Hapus File/Folder
6. Cari File/Folder
7. Kembali
8. Simpan ke Json
9. Keluar

```
=====
```

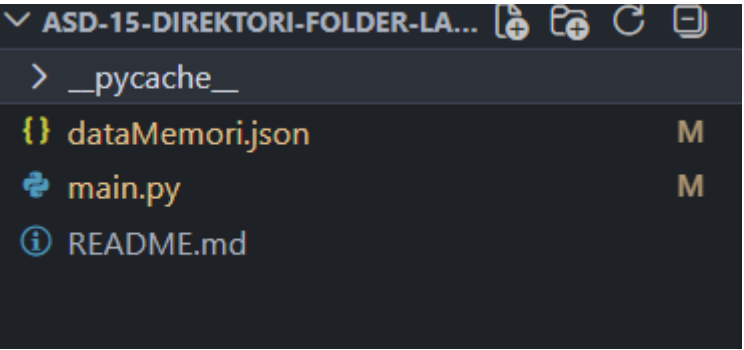
```
Pilih: 7
```

```
Path: home
```

```
Pilih: 8
Berhasil simpan data
```

```
Pilih: 9
Keluar dari program.
PS C:\laragon\www\ASD-15-Direktori-Folder-Laptop>
```

Lampiran C. Struktur Folder Proyek



Lampiran D. Commit History GitHub

Commits on Jun 26, 2026		
Link Video	inkAja committed 3 hours ago	e39a1e1
link Video	inkAja committed 12 hours ago	9675db4
Commits on Jun 17, 2026		
ubah	hl-ash-design committed last week	1359926
Commits on Jun 3, 2026		
hasil demo	hl-ash-design committed 3 weeks ago	9ad090d
delete accident duplicated codes	hl-ash-design committed 3 weeks ago	c381225
idk	hl-ash-design committed 3 weeks ago	18c050c
Add go in folder handling	hl-ash-design committed 3 weeks ago	6c4af68
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop	hl-ash-design committed 3 weeks ago	d74f7c9
ganti nama	inkAja committed 3 weeks ago	ea7b019
add go in folder handling	hl-ash-design committed 3 weeks ago	b3a267b
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop	inkAja committed 3 weeks ago	dd79058
ganti typo	inkAja committed 3 weeks ago	70262ea
add name handling for too long names	hl-ash-design committed 3 weeks ago	39852c3
delete comment	hl-ash-design committed 3 weeks ago	3c6031f
time	inkAja committed 3 weeks ago	8a136c8

Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 inkAja committed 3 weeks ago	ee25ab9	📄	<>
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 inkAja committed 3 weeks ago	bb37285	📄	<>
file testing 👤 hl-ash-design committed 3 weeks ago	eeec311	📄	<>
Masuk tambahan 👤 inkAja committed 3 weeks ago	b320e96	📄	<>
fix search output 👤 hl-ash-design committed 3 weeks ago	d538ae4	📄	<>
change search output 👤 hl-ash-design committed 3 weeks ago	7bca9ec	📄	<>
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 hl-ash-design committed 3 weeks ago	e0e0c3b	📄	<>
update time delay 👤 hl-ash-design committed 3 weeks ago	2d58e91	📄	<>
hapus update 👤 inkAja committed 3 weeks ago	16503ca	📄	<>
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 inkAja committed 3 weeks ago	5c48a48	📄	<>
update create 👤 inkAja committed 3 weeks ago	dbb51bc	📄	<>
update UI 👤 hl-ash-design committed 3 weeks ago	8822d36	📄	<>
json 👤 inkAja committed 3 weeks ago	e4ad2e6	📄	<>
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 inkAja committed 3 weeks ago	ea76499	📄	<>
Perbaikan untuk data ke json untuk parent, masukan nama biar bisa berulang, cari biar bisa munculin lokasi nya dimana(parentnya siapa) 👤 inkAja committed 3 weeks ago	dd19857	📄	<>
update pendataan waktu 👤 hl-ash-design committed 3 weeks ago	b91a0b4	📄	<>

melengkapi fitur tambah 👤 hl-ash-design committed 3 weeks ago	3603dc4	📄	<>
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 hl-ash-design committed 3 weeks ago	609ceff	📄	<>
update pendataan waktu dan cara akses json 👤 hl-ash-design committed 3 weeks ago	d771c11	📄	<>
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop 👤 inkAja committed 3 weeks ago	95e53af	📄	<>

Commits on Jun 3, 2026		
rapihin dikit	inkAja committed 3 weeks ago	682e550  
menambahkan pendataan waktu dan update display file/folder	hl-ash-design committed 3 weeks ago	a21ef35  
Commits on Apr 17, 2026		
Tambah json dan teman temannya	inkAja committed on Apr 17	760f24a  
Perubahan tampilan fitur 2(tambah nama current folder)	inkAja committed on Apr 17	aef91e9  
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop	inkAja committed on Apr 17	32fee7a  
perubahan di fitur buat file/folder	inkAja committed on Apr 17	9a680a5  
Delete Script.py	hl-ash-design authored on Apr 17	Verified 75a0257  
fix display files	hl-ash-design committed on Apr 17	1bd4073  
fix nama menu	inkAja committed on Apr 17	c9a5d64  
fix bug cek_child()	inkAja committed on Apr 17	00863a6  
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop	inkAja committed on Apr 17	dc54ac6  
tambahan di fitur masuk folder	inkAja committed on Apr 17	8962c65  
fix add folders/files	hl-ash-design committed on Apr 17	8f52091  
fix list menu	hl-ash-design committed on Apr 17	36d35e0  
Fix typo	inkAja committed on Apr 17	34f4749  
Merge branch 'main' of https://github.com/hl-ash-design/ASD-15-Direktori-Folder-Laptop	hl-ash-design committed on Apr 17	f34ae4a  
add change Directory function	hl-ash-design committed on Apr 17	4e6952b  
Add files via upload	AdamModric authored on Apr 17	Verified c19d026  
Commits on Apr 15, 2026		
menambahkan command	hl-ash-design committed on Apr 15	2a2e029  
Commits on Mar 21, 2026		
menambahkan module	hl-ash-design committed on Mar 21	666a371  
menambahkan segmen fitur yang ingin diimplementasikan	hl-ash-design committed on Mar 21	e14b172  
Commits on Feb 27, 2026		
Initial commit	hl-ash-design authored on Feb 27	Verified 5d6ccbb  
< Previous Next >		

Lampiran E. Source Code Utama

```
from datetime import datetime
```

```
import time
```

```
import json
```

```
import os
```

```
class Node:
```

```
    def __init__(self, name, tipe, mdtime=datetime.now().strftime("%d/%m/%Y %H:%M%p")):
```

```
        self.name = name
```

```
        self.tipe = tipe
```

```
        self.modtime = mdtime
```

```
        self.children = []
```

```
        self.parent = None
```

```
class stack:
```

```
    def __init__(self):
```

```
        self.data = []
```

```
    def push(self, item):
```

```
        self.data.append(item)
```

```
    def pop(self):
```

```
        if self.data:
```

```
            return self.data.pop()
```

```
        return None
```

```
def is_empty(self):  
    return len(self.data) == 0
```

```
# Mengubah metode akses json
```

```
script_dir = os.path.dirname(os.path.abspath(__file__))  
nama_file = os.path.join(script_dir, "dataMemori.json")
```

```
def json_to_node(data, parent=None):  
    if data:  
        node = Node(data['name'], data['tipe'], data['modtime'])  
        node.parent = parent  
        for child_data in data.get('children', []):  
            node.children.append(json_to_node(child_data, node))  
    return node
```

```
def bacaData(nama_file):  
    with open(nama_file, "r", encoding="utf-8") as f:  
        data_dict = json.load(f)  
    return json_to_node(data_dict)
```

```
class DirectoryTree:  
    def __init__(self):  
        self.root = bacaData(nama_file)
```

```
self.current = self.root
```

```
self.history = stack()
```

```
def cek_current(self):
```

```
    return self.current.name
```

```
# Fungsi Untuk menambah folder
```

```
def tambah(self):
```

```
    print("\n===Pilih tipe===")
```

```
    print("[1] FOLDER")
```

```
    print("[2] FILE")
```

```
def inputTipe():
```

```
    while True:
```

```
        pilihan = input("Pilih tipe (1/2): ")
```

```
        if pilihan == "1":
```

```
            return "Folder"
```

```
        elif pilihan == "2":
```

```
            return "File"
```

```
        else:
```

```
            print("Tipe salah. Silahkan input ulang")
```

```
tipe = inputTipe()
```

```
# Cek input tipe, apakah Folder atau bukan
```



```

if tipe not in ["Folder", "File"]:

    print("Tipe harus 'folder' atau 'file'")

    return

def inputnama():

    while True:

        nama = input(f'Masukan nama {tipe}: ')

        nama_terpakai = False

        # Cek apakah nama file/folder sudah digunakan

        for child in self.current.children:

            if child.name == nama:

                print("Nama sudah digunakan!, Silahkan masukan ulang")

                nama_terpakai = True

        if not nama_terpakai:

            return nama

nama = inputnama()

modtime = datetime.now().strftime("%d/%m/%Y %H:%M %p")

# simpan dengan nama,tipe Folder/File

node = Node(nama, tipe, modtime)

node.children = []

node.parent = self.current

self.current.children.append(node)

```

```

print(f'{tipe} berhasil dibuat.')

# Fitur menampilkan child(jika current bertipe folder)

def tampilan(self):

    if not self.current.children:

        print("Folder kosong")

        return

    # garis = " | " * level + " |—"

    sorted_children = sorted(self.current.children, key=lambda x: x.name.lower())

    print(f'===Isi Folder {self.current.name}===')

    for child in sorted_children:

        print(f'["  " if child.tipe.upper() == "FILE" else "  "] {child.name.ljust(25) if
len(child.name) < 25 else child.name[:22] + '...'} {child.modtime}')
```



```

def ubah_nama(self, nama_lama, nama_baru):

    cek = False

    for child in self.current.children:

        if child.name == nama_baru:

            print("Nama sudah digunakan!")

            return

    for child in self.current.children:

        if child.name == nama_lama:

            child.name = nama_baru
```

```
print("Berhasil mengganti nama.")
```

```
child.modtime = datetime.now().strftime("%d/%m/%Y %H:%M %p")
```

```
cek = True
```

```
if not cek:
```

```
    print("Folder/file tidak ditemukan")
```

```
def hapus(self):
```

```
    if not self.current.children:
```

```
        print("Folder kosong")
```

```
        return
```

```
    sorted_children = sorted(self.current.children, key=lambda x: x.name.lower())
```

```
    print(f"===Isi Folder {self.current.name}===")
```

```
    for child in sorted_children:
```

```
        print(f"[{" 📄 " if child.tipe.upper() == "FILE" else " 📁 "}] {child.name.ljust(25) if  
len(child.name) < 25 else child.name[:22] + '...'} {child.modtime}")
```

```
    nama = input("Masukan Nama yang ingin dihapus: ")
```

```
    for child in self.current.children:
```

```
        if child.name == nama:
```

```
            self.current.children.remove(child)
```

```
            print("Berhasil dihapus")
```

```
            return
```

```
    print("Folder/File tidak ditemukan.")
```

```
def cari(self, node, keyword):
```

```
    if not node:
```

```

return

# Validasi keyword agar case insensitive
if keyword.lower() in node.name.lower():

    parent_path = []

    parent_node = node.parent

    while parent_node:

        parent_path.append(parent_node.name)

        parent_node = parent_node.parent

    print("Ditemukan:", f"{node.name}" [Path:
    {'/'.join(reversed(parent_path))}/{node.name}"])

for child in node.children:

    self.cari(child, keyword)

def masuk(self):

    if not self.current.children:

        print("Folder kosong")

        return

    sorted_children = sorted(self.current.children, key=lambda x: x.name.lower())

    print(f"===Isi Folder {self.current.name}===")

    for child in sorted_children:

        print(f"[{ "📄 " if child.type.upper() == "FILE" else "📁 "} {child.name.ljust(25) if
len(child.name) < 25 else child.name[:22] + '...' } {child.modtime} ")

        nama = input("Masuk: ")

```

```

if self.current.children:

    for child in self.current.children:

        if child.name == nama and child.tipe == "Folder":

            self.history.push(self.current)

            self.current = child

            return

        elif child.name == nama and child.tipe == "File":

            print("Tidak bisa masuk ke file!")

            return

    print("Folder tidak ditemukan.")

```

```

def cek_child(self):

    print("\n", "Path: ", self.path())

    print("\n====Daftar Folder====")

    for child in self.current.children:

        if child.tipe == 'Folder':

            print(f[ 📁 ] {child.name}')

```

```

def cek_all_child(self):

    print("\n", "Path: ", self.path())

    print("\n====Daftar Folder====")

    for child in self.current.children:

```

```
print(f[{ " 📄 " if child.tipe.upper() == "FILE" else " 📁 " }] {child.name.ljust(25) if
len(child.name) < 25 else child.name[:22] + '...'}')
```

```
def kembali(self):
```

```
    prev = self.history.pop()
```

```
    if prev:
```

```
        self.current = prev
```

```
    else:
```

```
        print("Sudah di root")
```

```
def path(self):
```

```
    node = self.current
```

```
    path_list = []
```

```
    while node:
```

```
        path_list.append(node.name)
```

```
        node = node.parent
```

```
    return "/" .join(reversed(path_list))
```

```
def node_ke_dict(self, node):
```

```
    data = {
```

```
        "name" : node.name,
```

```
        "tipe" : node.tipe,
```

```
        "modtime" : node.modtime,
```

```
        "children" : [self.node_ke_dict(child) for child in node.children]
```

```
}
```

```
if node.parent is not None:
```

```
    data["parent"] = node.parent.name
```

```
return data
```

```
def simpan_json(self, filename):
```

```
    data_dict = self.node_ke_dict(self.root)
```

```
    with open(filename, "w", encoding="utf-8") as f:
```

```
        json.dump(data_dict, f, indent=4)
```

```
    print("Berhasil simpan data")
```

```
def main():
```

```
    tree = DirectoryTree()
```

```
    while True:
```

```
        print("\nPath:", tree.path())
```

```
        print("\nMenu:")
```

```
        print("1. Buat Folder/file")
```

```
        print("2. Lihat Isi Folder", tree.cek_current())
```

```
        print("3. Masuk ke Folder")
```

```
        print("4. Ganti Nama ")
```

```
        print("5. Hapus File/Folder")
```

```
print("6. Cari File/Folder")

print("7. Kembali")

print("8. Simpan ke Json")

print("9. Keluar")

print("=====")

pilihan = input("Pilih: ")


if pilihan == "1":

    tree.tambah()


elif pilihan == "2":

    tree.tampilan()


elif pilihan == "3":

    tree.masuk()


elif pilihan == "4":

    tree.cek_all_child()

    print()

    lama = input("Nama yang ingin diubah: ")

    baru = input("Nama baru: ")

    tree.ubah_nama(lama, baru)


elif pilihan == "5":
```



```
tree.hapus()
```

```
elif pilihan == "6":
```

```
    keyword = input("Masukkan kata kunci: ")
```

```
    tree.cari(tree.root, keyword)
```

```
elif pilihan == "7":
```

```
    tree.kembali()
```

```
elif pilihan == "8":
```

```
    tree.simpan_json(nama_file)
```

```
elif pilihan == "9":
```

```
    print("Keluar dari program.")
```

```
    break
```

```
else:
```

```
    print("Input tidak valid!")
```

```
time.sleep(2)
```

```
if __name__ == "__main__":
```

```
    main()
```